Instructions for Training and Exporting Datasets

The program used to train our models is called Roboflow. Roboflow is an end-to-end computer vision platform that enables developers and organisations to build, train, and deploy custom vision models efficiently. The platform streamlines the entire machine learning workflow, including data collection, annotation, model training, and deployment, making it accessible even for users with limited experience in artificial intelligence or computer vision.

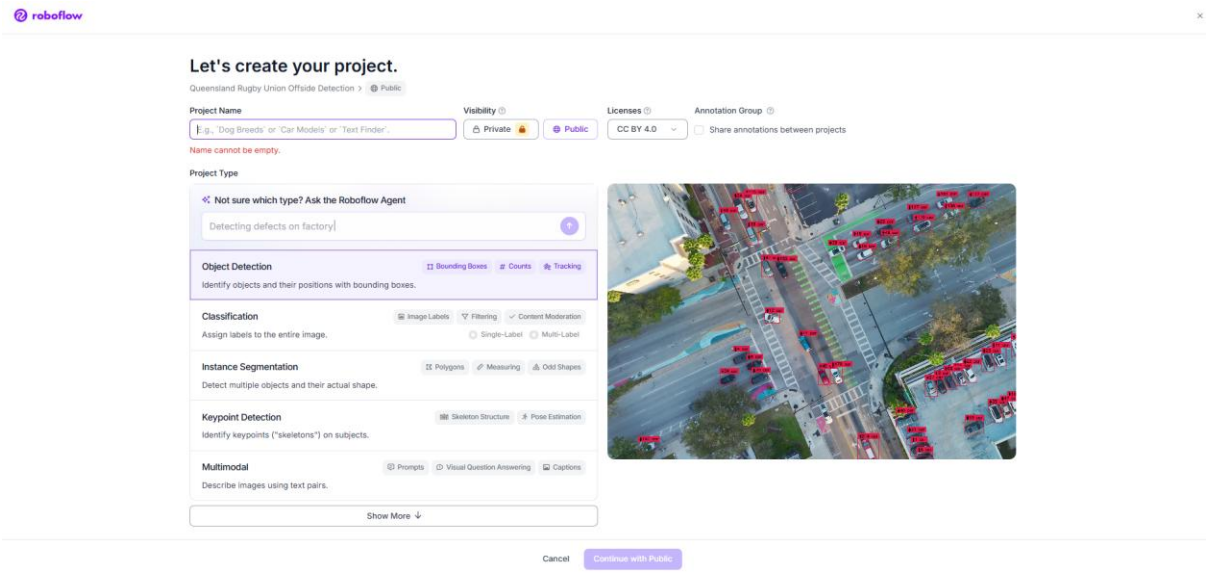
The free version of Roboflow provides a limited number of training credits each month. Additional credits normally require a paid subscription; however, alternative approaches can be used to reduce or bypass these training costs.

To generate our player detection model, along with the ruck, lineout, and ball detection models previously developed by earlier teams, Roboflow was used for dataset management, annotation, and model training. The following instructions will outline how you can achieve the same results for any models you wish to create or update.

Step 1. Create a new project

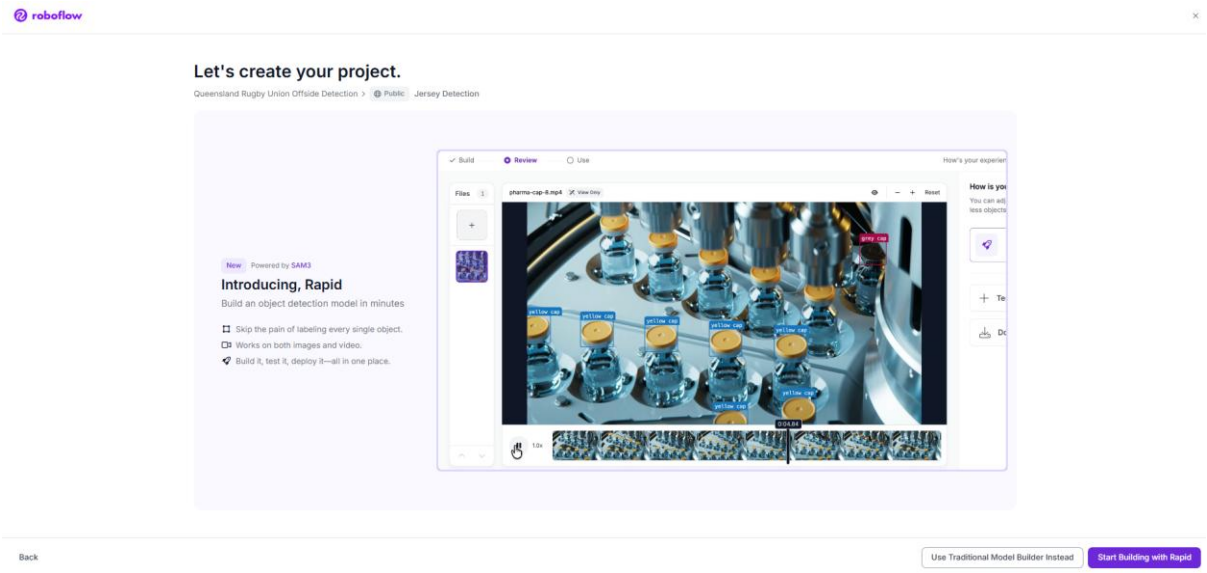
To begin, visit Roboflow.com, press ‘Get Started’ and sign up for a free account. Once signed in, create a new project and choose the project type most appropriate for the task being developed.

For this project, Object Detection was used, as the primary goal was to detect and localise entities such as players, the ball, rucks, and lineouts within video frames. However, other project types such as Classification or Segmentation may be more suitable for future extensions, depending on the specific computer vision task being explored.

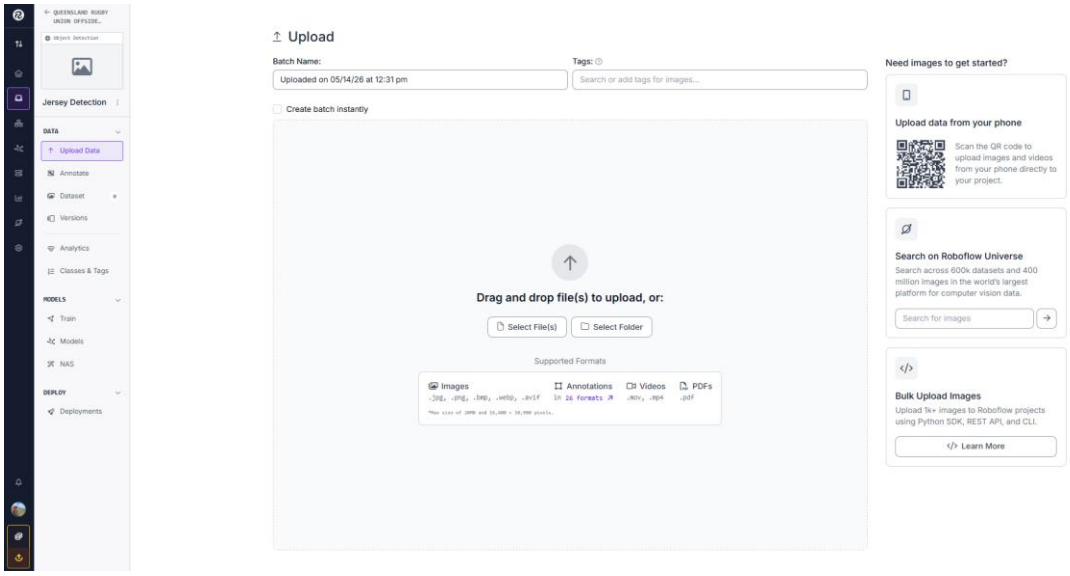


Roboflow may suggest that you try newer automated features or workflows when creating your project. For this project, the “Use Traditional Model Builder” option was selected, as it provided greater control and consistency throughout the training process.

However, many of Roboflow’s newer features may still be useful depending on the requirements of future work. It is also worth noting that the platform will continue recommending additional tools and workflows throughout project development.

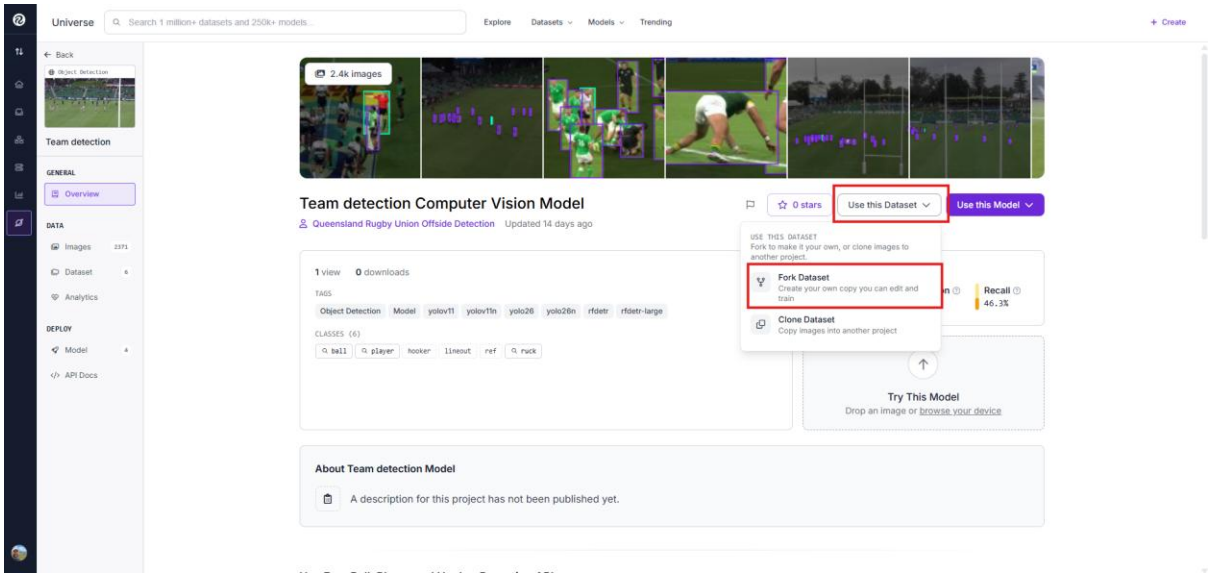


You are now at the main project page:



Step 2: Getting previous dataset to your Roboflow project.

This Github link will provide you with the link to the previous project in Roboflow. To get these datasets, click the ‘Use this Dataset’ button and then click ‘Fork Dataset’ (You must be signed into Roboflow).



Select the project you wish to import or download the dataset into (if required). Once completed, a separate copy of the dataset and model configuration will be available within your own workspace, allowing you to modify, annotate, and train it independently.

Step 3. Uploading images for annotation

Once the project has been created, additional images can be added to the dataset over time to further improve model accuracy and robustness. To begin adding data, select “Upload Data” and upload either individual images or video files that you wish to include in the dataset.

It should be noted that Roboflow’s upload and dataset management workflow can occasionally be cumbersome, particularly when removing poor-quality or unwanted frames. For this reason, it is recommended to use the `frame-extractor.py` script provided in the GitHub repository. This allows frames to be extracted and reviewed locally before being uploaded to Roboflow, significantly improving dataset quality and reducing manual cleanup work.

```
# === CONFIG ===
input_root = r"\Where\Your\Videos\Are"
output_folder = r"\Where\You\Want\To\Save\Frames"
frame_interval_seconds = 2
max_workers = 4
```

Set `input_root` to the directory containing your video files. The script searches recursively, meaning it will automatically process videos located within all subfolders of the specified directory.

Set `output_folder` to the location where the extracted frames should be saved. Frames are organised into separate folders corresponding to each processed video file.

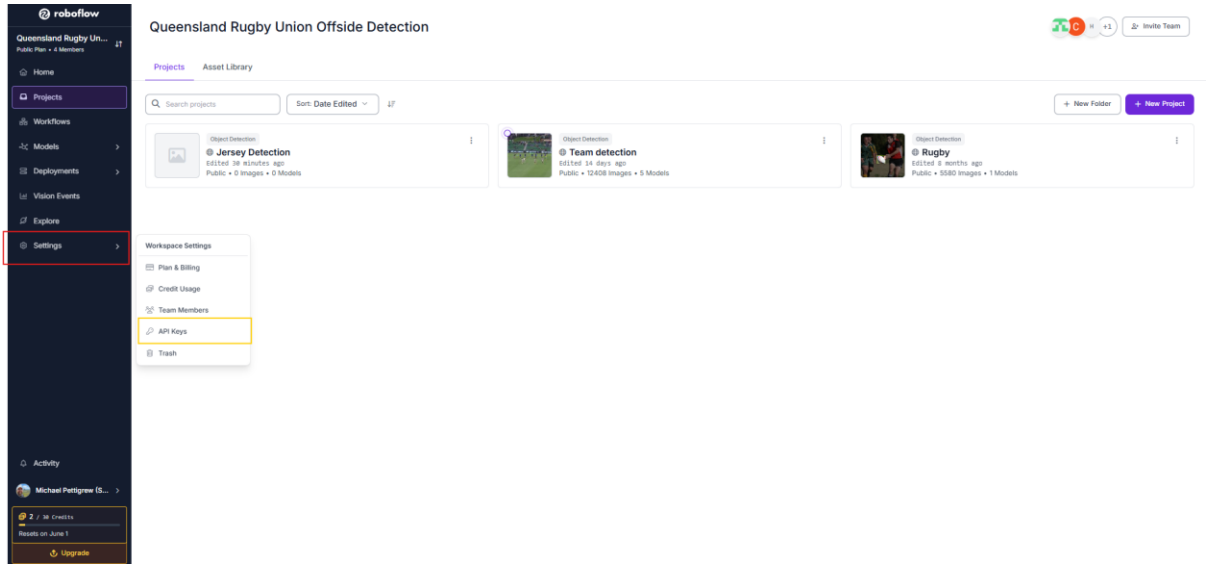
The `frame_interval_seconds` variable controls how frequently frames are extracted from the video. For example, a value of 2 will save one frame every two seconds.

Once your videos are processed into images, you can then upload to Roboflow. If you have lots of images, then it’s recommended to use the `roboflow-uploader.py` in the GitHub repository.

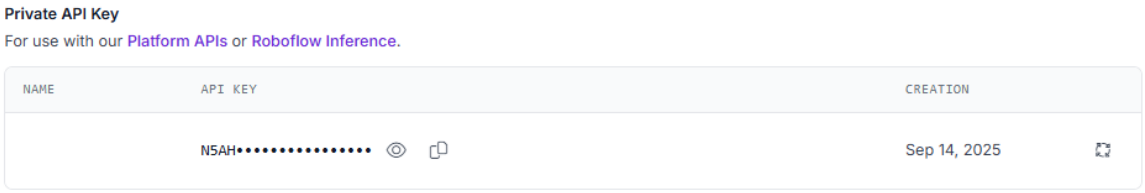
It is recommended to go through your images before uploading in order to remove useless frames.

3.1 roboflow-uploader.py

To use `roboflow-uploader.py` you will need your Roboflow API key, click settings and 'API keys'



Retrieve your private API key from the Roboflow dashboard. This key should be kept confidential and never committed directly into source code or public repositories.



For security purposes, it is recommended to store the API key using environment variables or a separate configuration file that is excluded from version control.

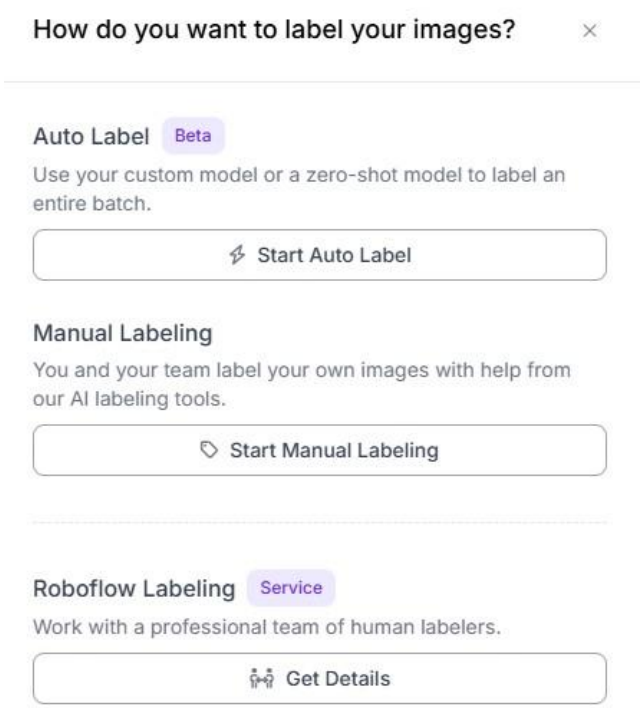
```
API_KEY = os.getenv("ROBOFLOW_API_KEY")

WORKSPACE = "Your workspace"
PROJECT = "Your project"

INPUT_ROOT = r"D:\your\images"
```

In the Roboflow URL, the workspace name is shown in the section highlighted in red, while the project name is shown in the section highlighted in yellow. Direct `INPUT_ROOT` to the root folder of your images and run the program, it should upload everything smoothly.

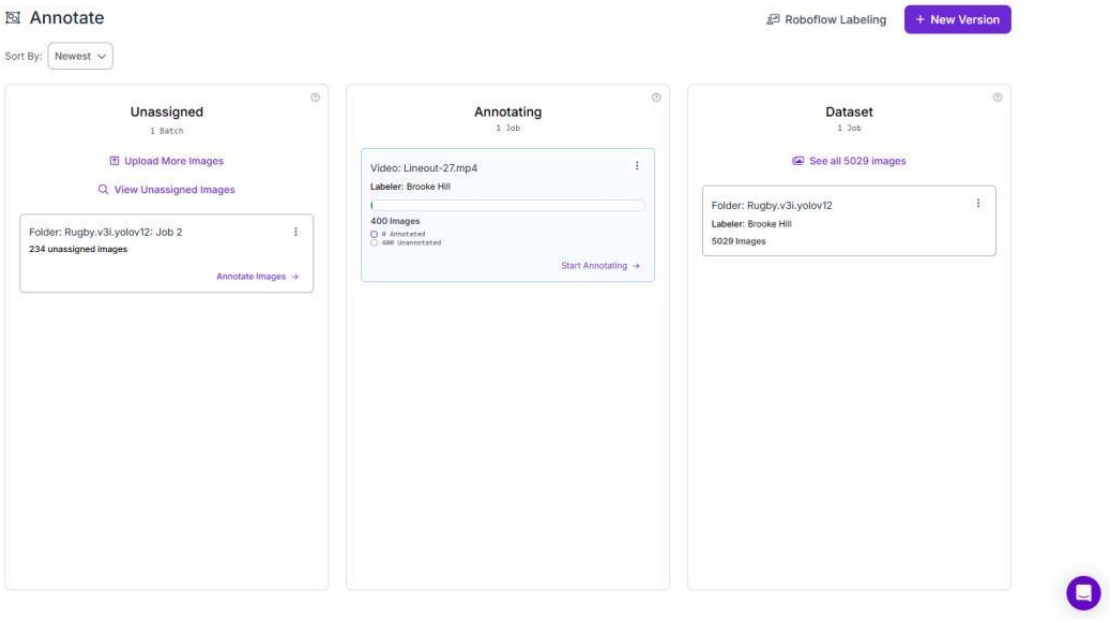
Step 4. Annotating Images

There are a couple of ways you can annotate images - manually or by using a Roboflow model to assist with annotations.

When prompted with a “How do you want to label your images?” question, select “Manual Labelling”.



Select “Annotating” in the centre column:



Click the first image and the annotation editor will open.

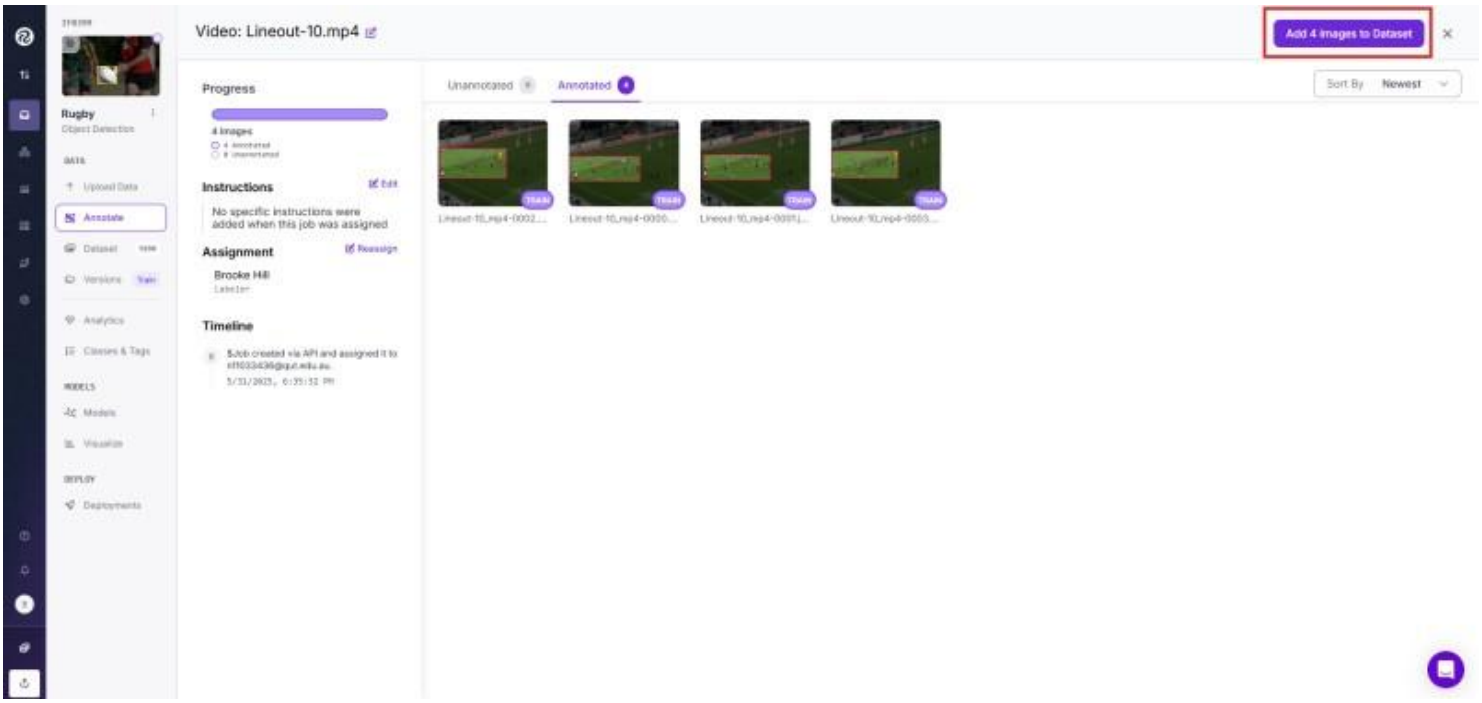
Draw a box around the point you want to annotate. In this instance it is the referee:

Repeat these manual annotations for the remainder of the images, dragging and drawing boxes around the referee. You can add multiple annotations to the one image, when appropriate.

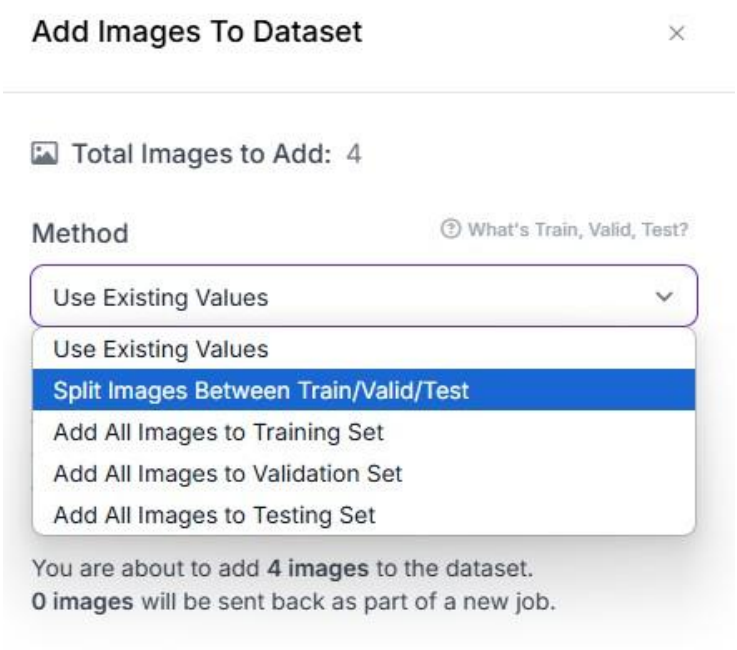
Step 5: Add Annotated Images to Dataset

Once you are satisfied with the annotations, you can add them to an existing dataset.

To add images into the dataset, navigate to the Annotated images tab and in top right corner select Add X Images to Dataset.



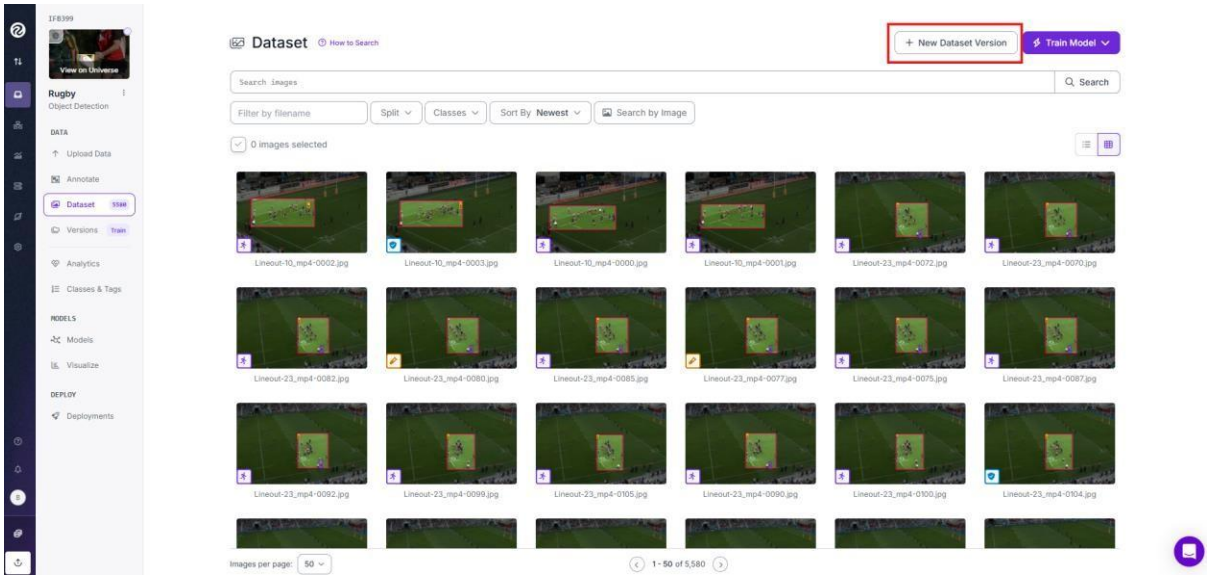
Next, you want to select where you want those images to be added to. Generally, we chose to Split Images Between Train/Valid/Test option.



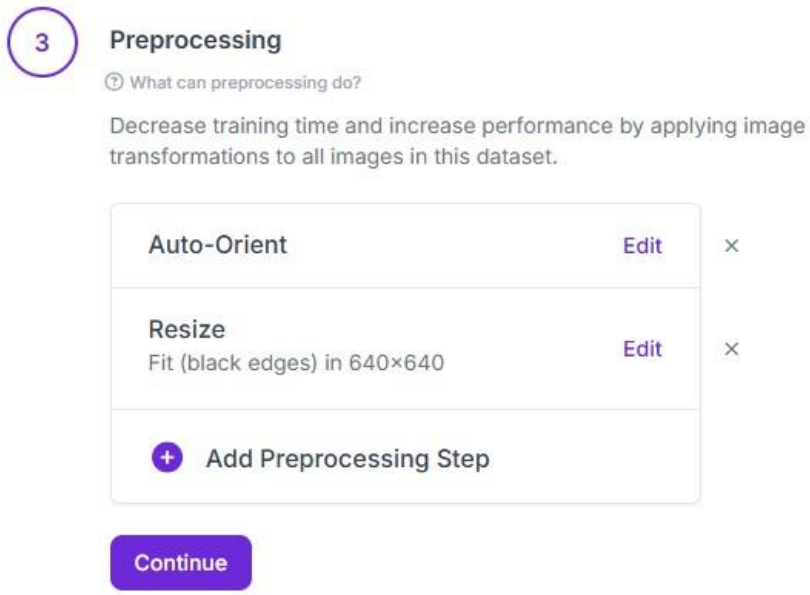
After you have selected the location to add the images to, press “Add Images”, this will move the images into your dataset.

Step 6: Creating a Dataset

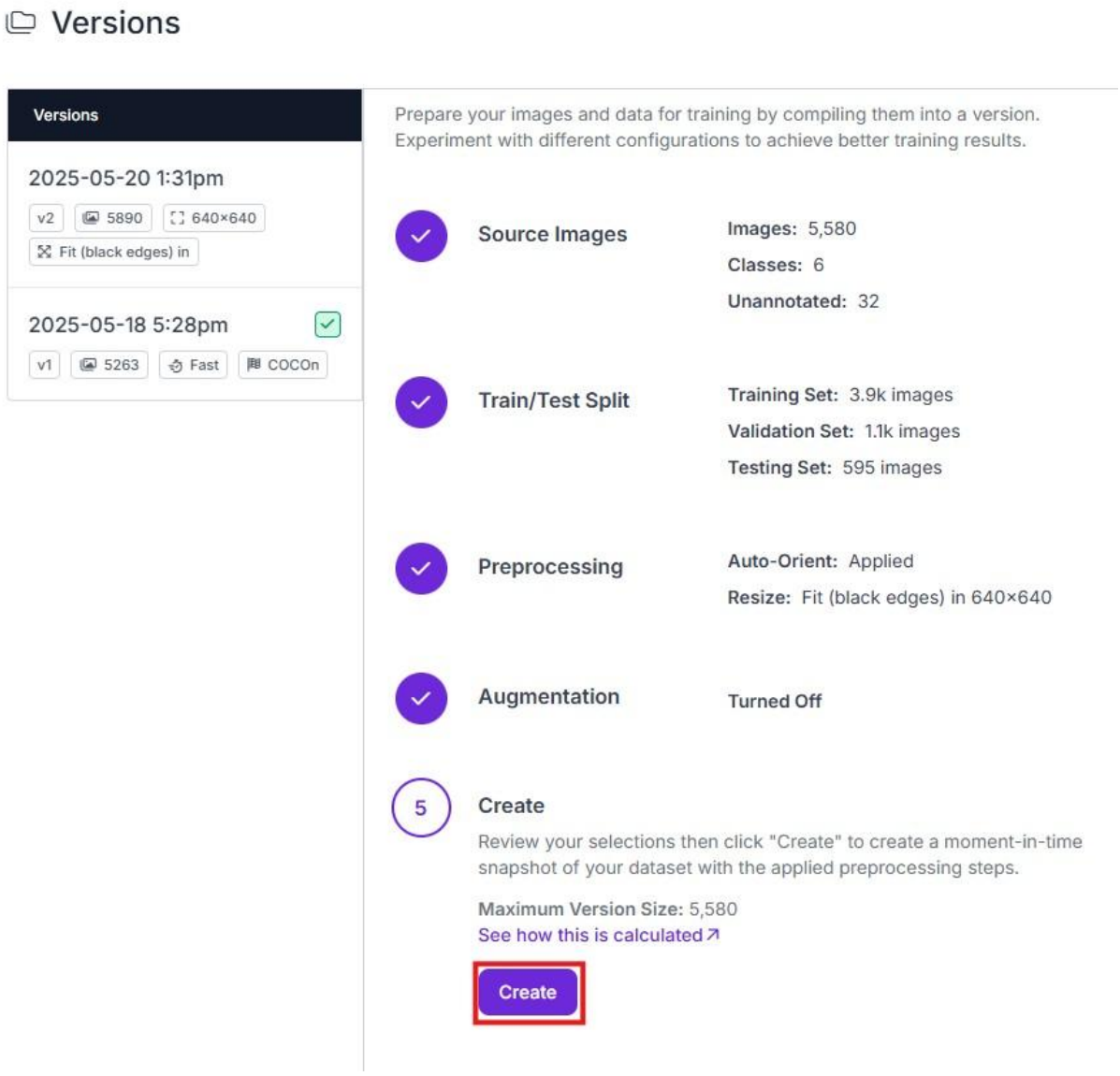
After you are happy with your dataset, you can create a New Dataset Version to use for model training.



On the next page, select the features you want for your version. For this, we used the standard 640x640 size for all images (fit within this size with black edges). Make sure not to select the “Reflect Edges” mode, as it produced undesirable outputs.



After selecting all wanted preprocessing, select “Create”:



Congratulations, you have now made a dataset version.

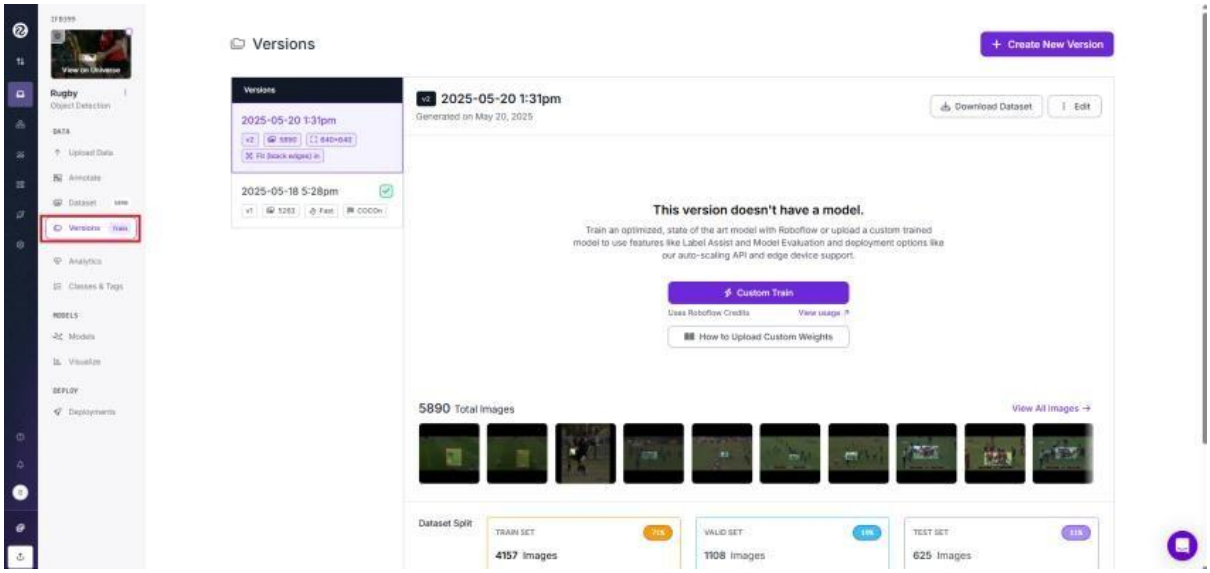
Step 7: Training the model

Roboflow offers the option of training the model, this does cost credits so you can run out very quickly. Instead, it is suggested that you setup a Coogle Colab account.

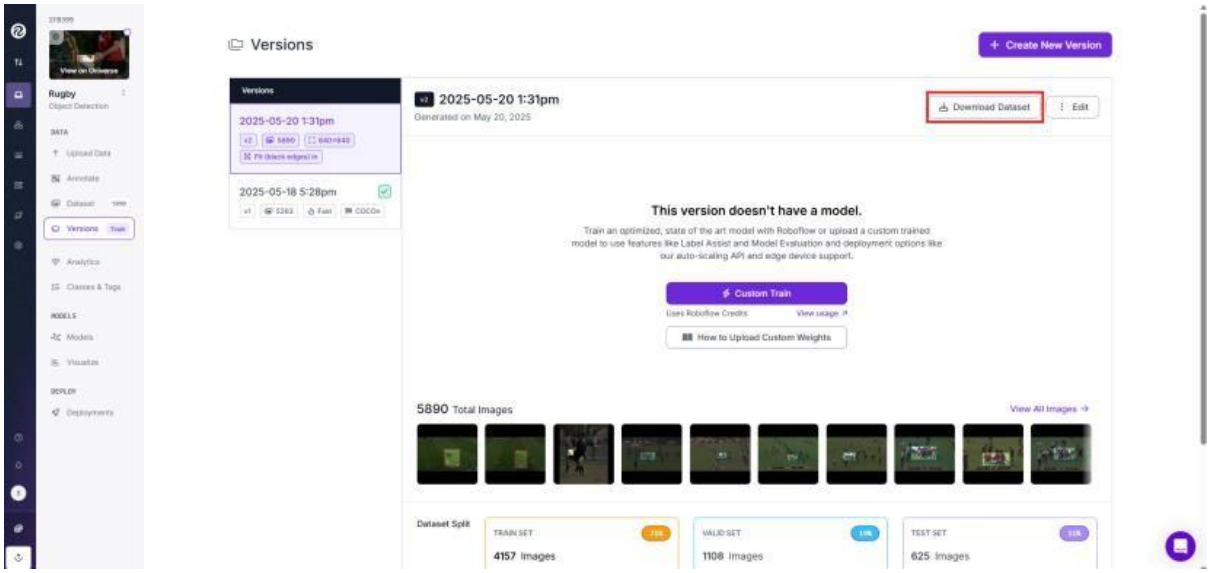
7.1 Retrieving your Dataset

To acquire an executable code summary which allows you to train the dataset externally, first ensure that you have created a dataset version, following the steps outlined in step 6.

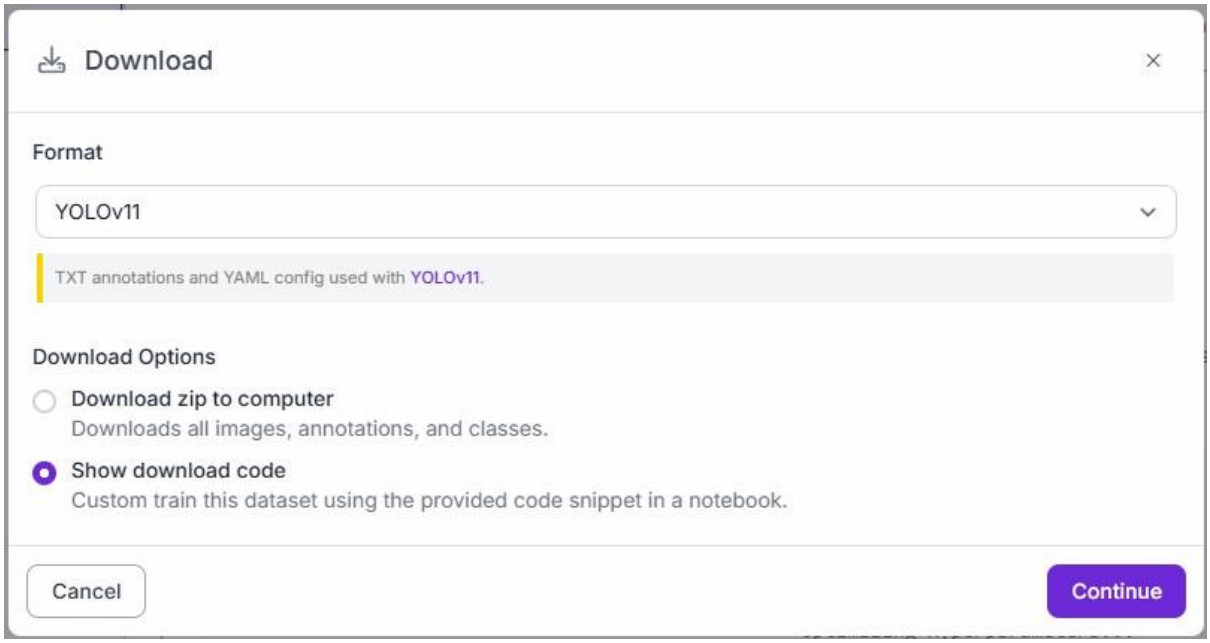
Once this step is complete, click on the Versions tab located on the left-hand side menu:



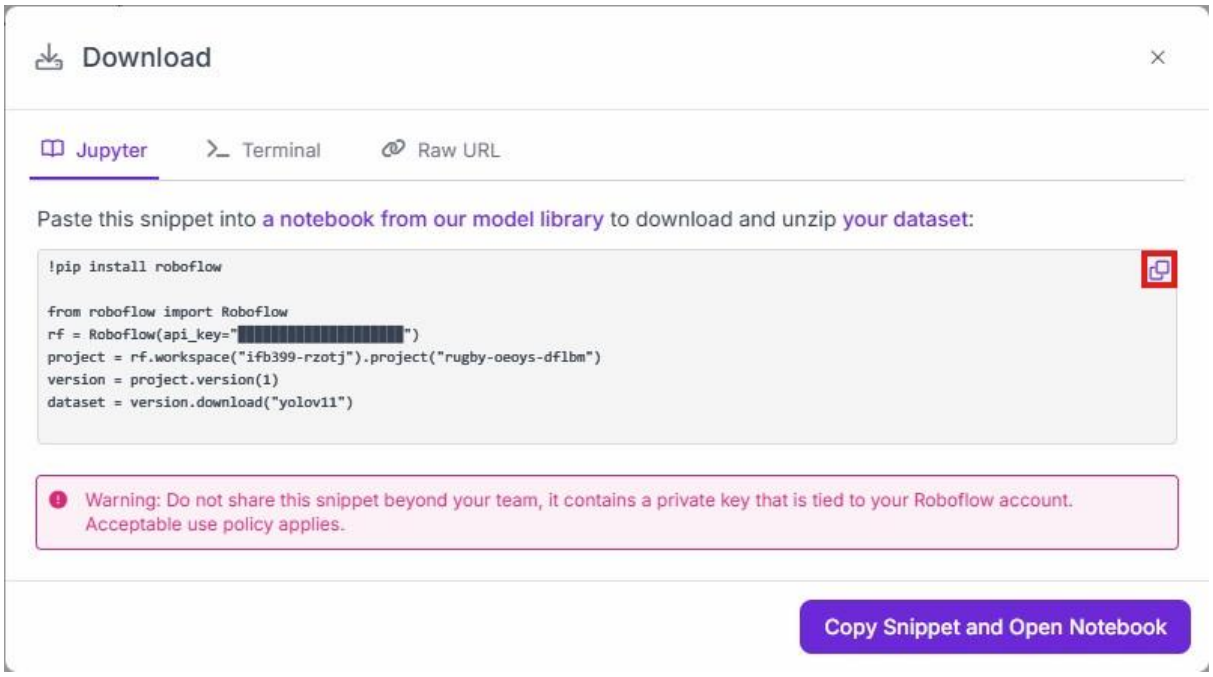
Next, select “Download Dataset”.



After this, select the downloaded code format and click “Show Download Code”. Select “Continue”



Select the copy button on the top right-hand side of where the code is.



7.2 Google Colab Training

Located in the GitHub repository is model-trainer.py copy the code and go to Google Colab. Once signed-in, select 'New notebook'. At the top, select the Runtime tab, then 'Change Runtime Type'. Select Python 3 in the runtime type drop-down menu, then select the T4 GPU as the hardware accelerator and paste the copied code.

```
####################################

  #PASTE YOUR ROBOFLOW SNIPPET HERE#

####################################

from ultralytics import YOLO
# keep your model
model = YOLO("yolo26m.pt")

model.train(
    data=f"{dataset.location}/data.yaml",
    epochs=100,
    imgsz=640,
    batch=16
)
```

Paste your roboflow code you got from part 7.1 and then click the runtime "Run all" near the top.

```
Starting training for 100 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      1/100      8.94G      1.379      2.823   0.005995         52        640: 100% ──────────── 14/14 1.9s/it 26.7s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 1/1 3.5s/it 3.5s
                   all         10        215      0.923      0.233      0.384      0.219

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      2/100        9G      1.429      1.814   0.005386         51        640: 100% ──────────── 14/14 1.6it/s 9.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 1/1 7.3it/s 0.1s
                   all         10        215      0.482       0.58      0.596      0.398

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      3/100      9.17G      1.462      1.559   0.005678        106        640: 100% ──────────── 14/14 1.6it/s 8.8s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 1/1 7.5it/s 0.1s
                   all         10        215      0.554      0.323      0.461       0.25
```

The cell will usually take 20-30 minutes to run. It is recommended to keep the page open while it is running, as the free version of Colab does not allow background processing.

To obtain the best results, you want the training to run until convergence. This means it will run until the performance plateaus. The easiest way to monitor this is by checking the mAP50 value in each epoch. If it remains the same after many epochs, it has likely converged.

Once training is complete, if the model has not converged, it may be worth training for a longer period to obtain better performance. To run training for longer, increase the number of epochs in the yolo train line and run the cell again by selecting the play button to the left.

When training is done, download your model.